

Algorithmization and Programming of Solutions

Part I

TOPIC #7

PROCESSING OF ARRAYS: SEARCH AND SORT ALGORITHMS

RTU LECTURER OLGA YAKOVLEVA, MG.SC.ING

Table of contents

1. Processing of arrays - search algorithms:
 - linear search
 - linear search with sentinel
 - binary search
2. Processing of arrays - sort algorithms:
 - bubble sort
 - insertion
 - selection
3. Practical work #3: Programming of search and sort algorithms in programming language C (for arrays)

1. Processing of arrays – search algorithms

LINEAR SEARCH, LINEAR SEARCH WITH SENTINEL, BINARY SEARCH

Search algorithms

The search problem: locate an element x in a list of distinct elements $a^1, a^2, \dots, a^n$ or determine that it is not in the list

The solution of the search problem: the location of the element in the list that equals x or 0 if such element is not in the list

Search algorithms are designed to solve the search problem, i.e., to search for a specified element in a data structure

Examples of search algorithms:

- **linear search**
- **linear search with sentinel**
- **binary search**

Model of linear search

Picture original source
<http://www.oxfordmathcenter.com/drupal7/node/31>

Key	List
3	6 4 1 9 7 3 2 8
3	6 4 1 9 7 3 2 8
3	6 4 1 9 7 3 2 8
3	6 4 1 9 7 3 2 8
3	6 4 1 9 7 3 2 8
3	6 4 1 9 7 3 2 8

Processing of one-dimensional array – Linear search, example I (1/3)

Implementation in C

```
#include <stdio.h>
#include<stdbool.h>
#define LENGTH 9
int main() {
    int a[] = {1,38,2,9,12,2,4,20,17};
    int x, i;
    bool f = false;
    printf("Input an element value you want to
           search for: ");
    scanf("%d", &x);
```

Processing of one-dimensional array – Linear search (2/3)

Implementation in C

```
//search for the first element in the array a  
that is equal to x  
for (i = 0; i < LENGTH; i++) {  
    if (a[i] == x) {  
        f = true;  
        break;  
    }  
}
```

Processing of one-dimensional array – Linear search (3/3)

Implementation in C

```
if (f) printf("\nElement %d is found at  
           position with index %d.\n", x,  
           i);  
else printf("\nElement %d is not found.\n",  
           x);  
return 0;  
}
```

Processing of one-dimensional array – Linear search, example II (1/2)

Implementation in C

```
#include <stdio.h>
#include<stdbool.h>
#define LENGTH 9
int main() {
    int a[] = {1,38,2,9,12,2,4,20,17};
    int x, i;
    bool f = false;
    printf("Input an element value you want to
           search for: ");
    scanf("%d", &x);
    //search for the first element in the array a
    that is equal to x
```

Processing of one-dimensional array – Linear search, example II (2/2)

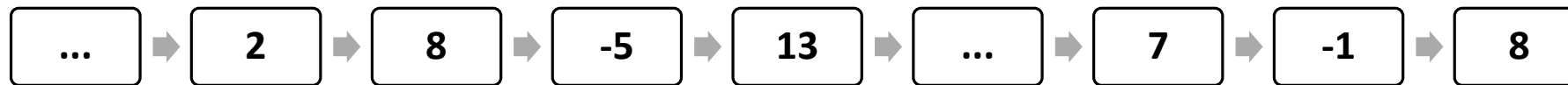
Implementation in C

```
for (i = 0; i < LENGTH; i++) {  
    if (a[i] == x) {  
        printf("\nElement %d is found at  
            position with index %d.\n", x, i);  
        f = true;  
    }  
}  
if (!f) printf("\nElement %d is not found.\n",  
                x);  
return 0;  
}
```

Model of linear search with sentinel value

$x=8$

sentinel



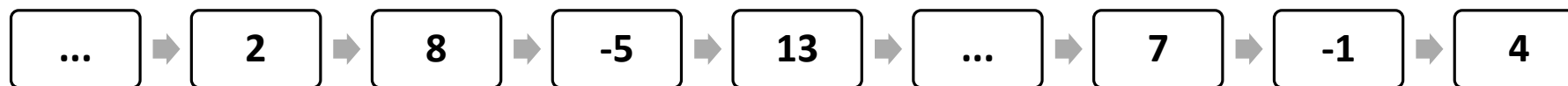
$x==2?$

$x==8?$

Stop! Found!

$y=4$

sentinel



$y==2?$

$y==8?$

$y==-5?$

and so on...

Stop! Found! But it's
out of range!

Processing of one-dimensional array – Linear search with sentinel value (1/2)

Implementation in C

```
#include <stdio.h>
#define LENGTH 9
int main() {
    int a[] = {1, 38, 2, 9, 12, 2, 4, 20, 17};
    int x, i = 0;
    printf("Input an element value you want to  
search for: ");
    scanf("%d", &x);
    a[LENGTH] = x; //setting a sentinel value
```

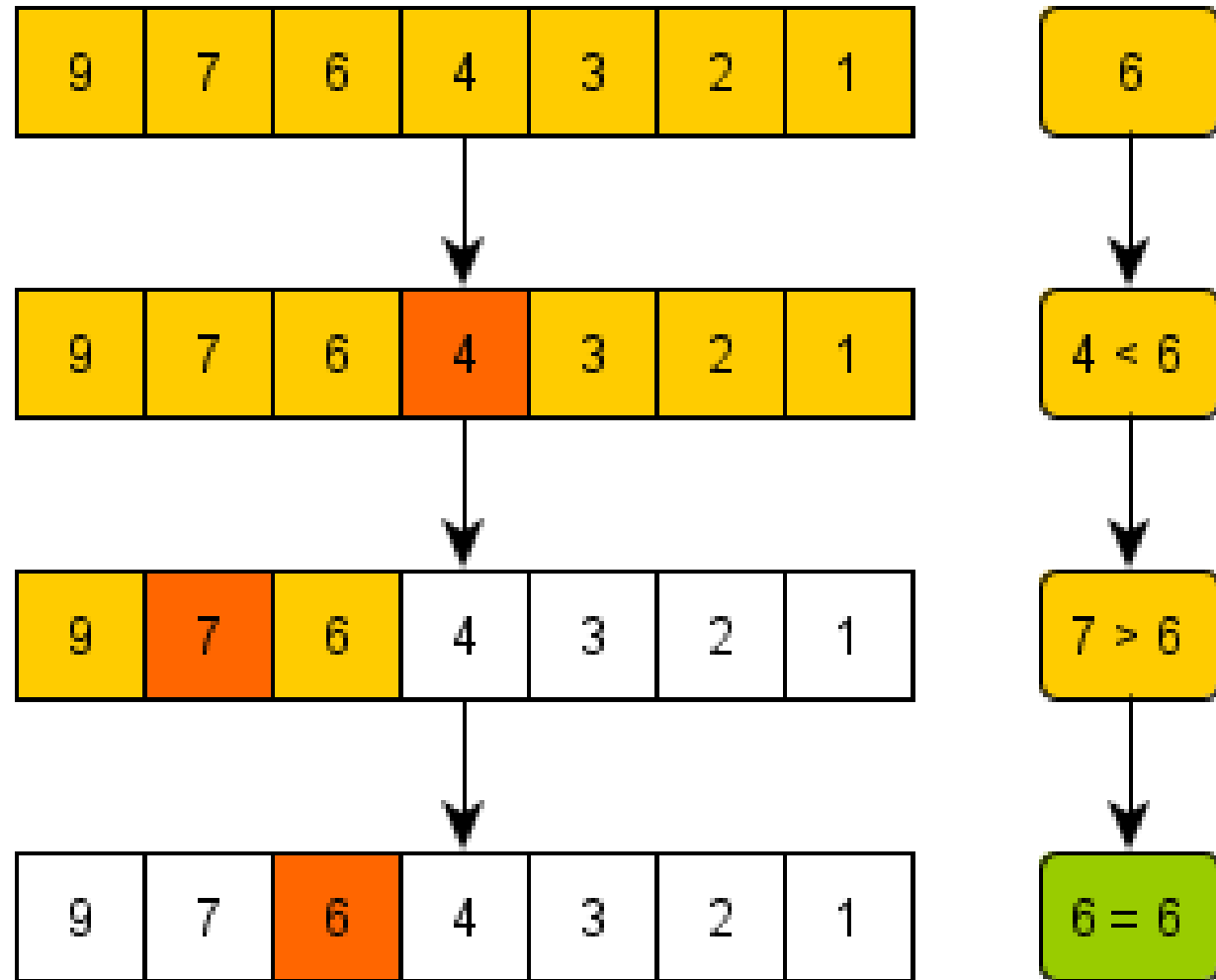
Processing of one-dimensional array – Linear search with sentinel value (1/2)

Implementation in C

```
//search for the first element in the array a
//that is equal to x
while (a[i] != x) i++;
if (i == LENGTH) printf("\nElement %d is not
                        found.\n", x);
else printf("\nElement %d is found at position
            with index %d.\n", x, i);
return 0;
}
```

Model of binary search, is usable only for sorted data structures!!!

Picture original source
<http://en.algoritmy.net/article/40119/Binary-search>



2. Processing of arrays – sort algorithms

BUBBLE SORT, INSERTION, SELECTION

Sort algorithms

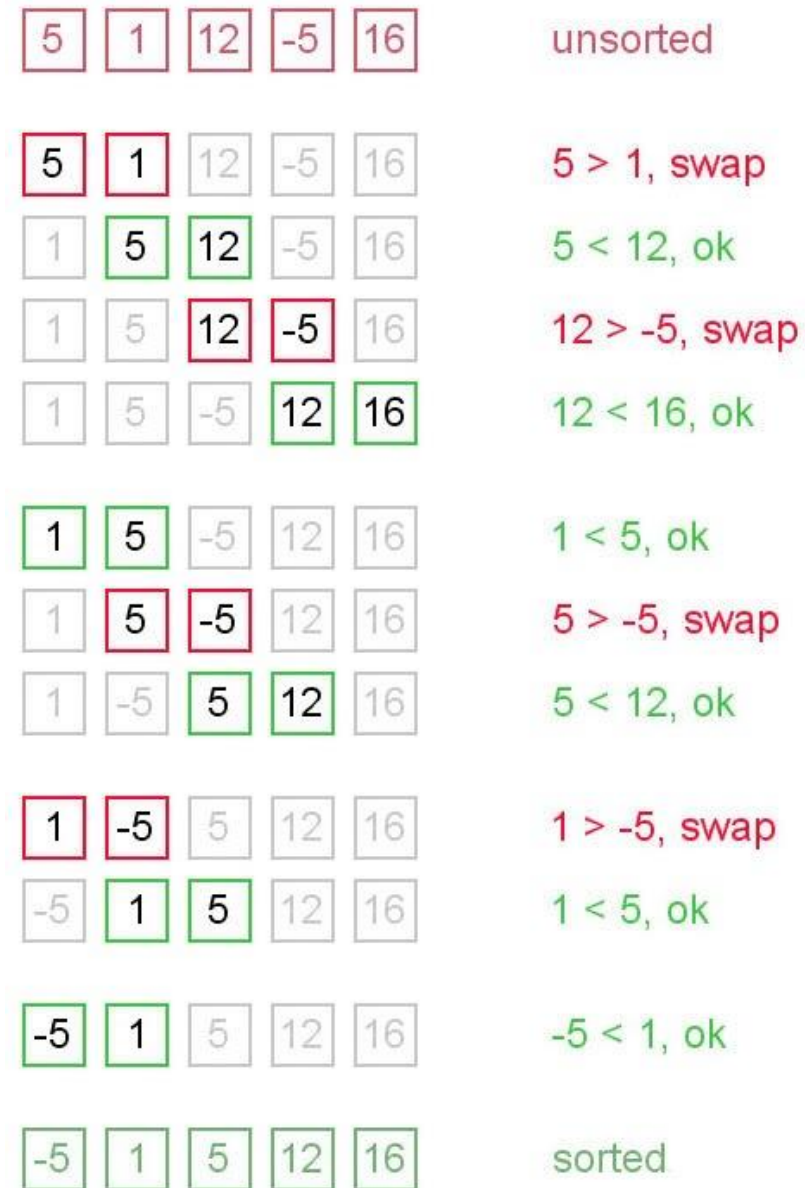
The sort problem: elements in a data structure are not ordered according to a specified criteria, e.g., Z to A, ascendingly

The solution of the sort problem: elements in the data structure are sorted according to the specified criteria

Sort algorithms are designed to solve the sort problem, i.e., to rearrange elements in a data structure in order to get a sorted sequence

Model of bubble sort

Picture original source
<http://www.java2novice.com/java-sorting-algorithms/bubble-sort/>



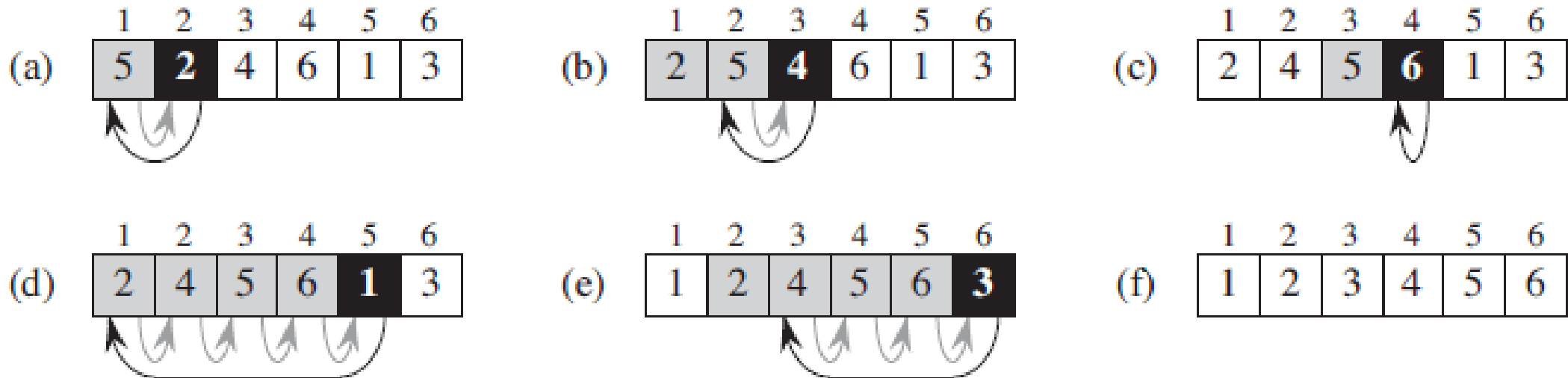
Bubble sort

Pseudocode

```
For I = 0 to N - 3
  For J = 0 to N - 2
    If (A(J) > A(J + 1))
      Temp = A(J)
      A(J) = A(J + 1)
      A(J + 1) = Temp
    End-If
  End-For
End-For
```

Model of insertion

Picture original source <http://www.java2novice.com/java-sorting-algorithms/insertion-sort/>



Insertion

Pseudocode

For $I = 1$ to $N-1$

$J = I$

 Do while $(J > 0)$ and $(A(J) < A(J - 1))$

$Temp = A(J)$

$A(J) = A(J - 1)$

$A(J - 1) = Temp$

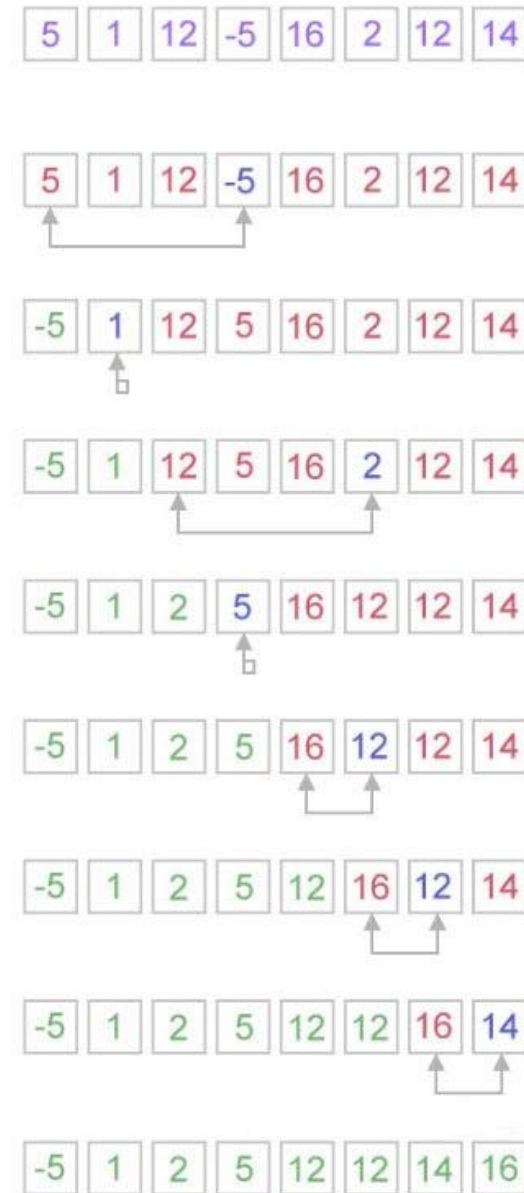
$J = J - 1$

 End-Do

End-For

Model of selection

Picture original source
<http://www.java2novice.com/java-sorting-algorithms/selection-sort/>



Selection

Pseudocode

```
For I = 0 to N-1 do:  
  Smallsup = I  
  For J = I + 1 to N-1 do:  
    If A(J) < A(Smallsup)  
      Smallsup = J  
    End-If  
  End-For  
  Temp = A(I)  
  A(I) = A(Smallsup)  
  A(Smallsup) = Temp  
End-For
```

3. Practical work #2

PROGRAMMING OF SEARCH AND SORT ALGORITHMS IN PROGRAMMING
LANGUAGE C (FOR ARRAYS)

Programming of search and sort algorithms in programming language C (for arrays)

1. Create a program that implements a binary search algorithm and processes a one-dimensional array, which consists of 10 sorted elements

OR

2. Create a program that implements a selection algorithm and processes two-dimensional array, which consists of 3 rows and 4 columns, e.g.,

Before sorting

16	15	18	21
20	11	10	19
12	17	14	13



10	11	12	13
14	15	16	17
18	19	20	21

After sorting